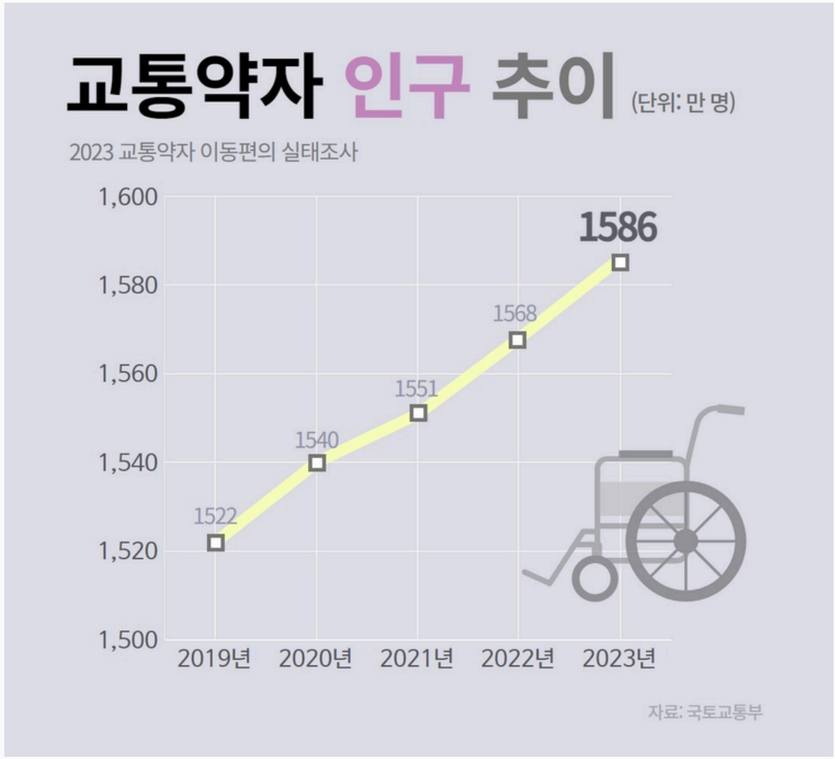
ROS2와 Gazebo 기반 버스 승하차 보조 로봇

이서연

문제 정의



출처: 국토교통부

- 고속버스터미널에서는 여러 노선과 승차홈이 동시에 운영되기 때문에 이용자가 자신이 타야 할 버스의 위치를 정확히 찾기 어렵다.
- 기존 안내 방식은 전광판, 표지판, 모바일 예매 앱처럼 정보 제공 중심으로 이루어져 있어서 사용자는 이를 직접 해석하고 이동 경로를 판단해야 한다.
- 특히 증가하고 있는 교통약자인 고령자를 포함한 초행 이용자, 짐이 많은 승객은 복잡한 터미널 구조 속에서 승차홈 위치를 빠르게 파악하기 어렵고, 출발 시간이 가까운 상황에서는 이동 부담과 혼란이 커질 수 있다.

Solution

프로젝트 설명

- 버스를 이용하는 승객의 탑승을 보조하는 자율이동로봇 시스템
- ROS2 기반으로 설계,구현
- 노약자 등 대중교통 이용에 어려움을 겪는 승객을 대상으로, 원하는 버스 노선(색상으로 구분)을 선택하면 로봇이 해당 버스 정류장까지 자율 주행하여 안내

프로젝트 개요

항목	내용
플랫폼	ROS2 Humble + Gazebo Fortress (시뮬레이션)
로봇 형태	차동 구동 (Differential Drive) 이동 로봇
센서	2D LiDAR (360°), RGB 카메라 (640×480)
핵심 기능	버스 색상 인식, 자율 경로 계획 및 주행, 장애물 회피, GUI 기반 사용자 인터페이스
내비게이션	Nav2 스택 (AMCL 기반 위치 추정, BehaviorTree 경로 계획)

- 시뮬레이션 환경에는 빨강(Red), 파랑(Blue), 초록(Green), 노랑(Yellow) 4개 색상의 버스가 정의되어 있고, 각 버스는 고유한 정류장 위치를 가진다.
- 사용자가 GUI에서 버스를 선택하거나 카메라가 특정 색상의 버스를 감지하면, 로봇이 해당 정류장으로 자율 주행한다.

시스템 구조

```
[Gazebo 카메라] ──→ /camera/image_raw ──→ [Perception Node]
                                                ├──→ HSV 색상 감지
                                                └──→ /bus_color

[GUI 버튼 클릭] ──→ /selected_bus ──────────→ [Navigation Node]
                                                ├──→ 상태 머신 관리
                                                ├──→ Nav2 액션 서버 호출
                                                └──→ /robot_state
                                                     /display_text
                                                     /goal_marker

[Gazebo LiDAR] ──→ /scan ──────────────────→ [Safety Node]
                                                ├──→ 전방 FOV 스캔
                                                └──→ /obstacle_detected
                                                     /display_text

[Gazebo Odometry] ──→ /odom ──────────────→ [OdomTF Broadcaster]
                                                └──→ /tf (odom→base_link)

                                          [Bus Markers Node]
                                                └──→ /bus_markers (RViz 시각화)

[Display Node (Tkinter GUI)]
  ├─ 수신: /display_text, /robot_state
  ├─ 발행: /selected_bus (버튼 클릭 시)
  └─ 표시: 로봇 상태, 안내 메시지
```

ROS 노드 구성

노드명	실행 파일	역할
perception_node	perception_node.py	RGB 카메라 영상을 HSV 색상 공간으로 변환하여 버스 색상 감지
navigation_node	navigation_node.py	상태 머신으로 로봇 이동 상태를 관리하고 Nav2에 주행 목표 전송
safety_node	safety_node.py	LiDAR 스캔 데이터로 전방 장애물 감지 및 경고 발행
display_node	display_node.py	Tkinter 기반 GUI로 버스 선택 인터페이스 및 상태 표시 제공
odom_tf_broadcaster	odom_tf_broadcaster.py	/odom → /tf 변환 발행 (Gazebo 서버 모드 보완)
bus_markers_node	bus_markers_node.py	각 버스 정류장 위치를 RViz에 3D 마커로 시각화

구현 방법

3.1 버스 색상 인식 (perception_node.py)
HSV 기반 색상 임계값 탐지 + 모폴로지 필터링

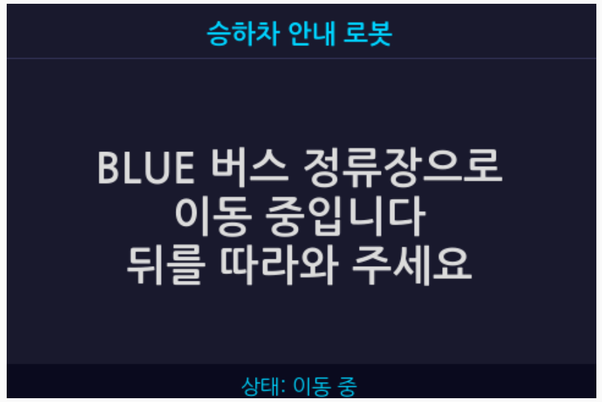
- **색상별 HSV 범위:**
  Red: H 0-10/170-180, S 120-255, V 70-255
  Blue: H 100-130, S 100-255, V 50-255
  Green: H 40-80, S 60-255, V 50-255
  Yellow: H 20-35, S 100-255, V 100-255
- 이전 색상과 다를 때만 퍼블리시

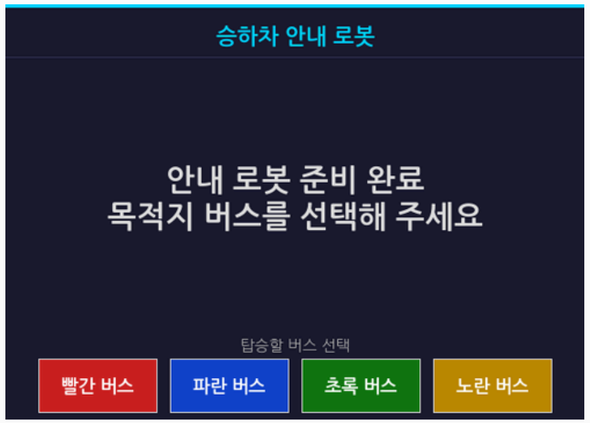
3.2 자율 내비게이션 (navigation_node.py)
상태: IDLE → NAVIGATING → ARRIVED
- **버스 정류장 좌표**:
  Red (5.5, -2.5, 90°) / Blue (3.0, 3.0, 90°)
  Green (0.0, 3.0, 90°) / Yellow (-5.5, -2.5, 90°)
- Nav2 NavigateToPose 액션 사용

3.3 장애물 감지 (safety_node.py)
- 전방 ±30° LiDAR 포인트 사용
- 0.8m 이내 장애물 시 경고 발행
- navigating 상태일 때만 활성

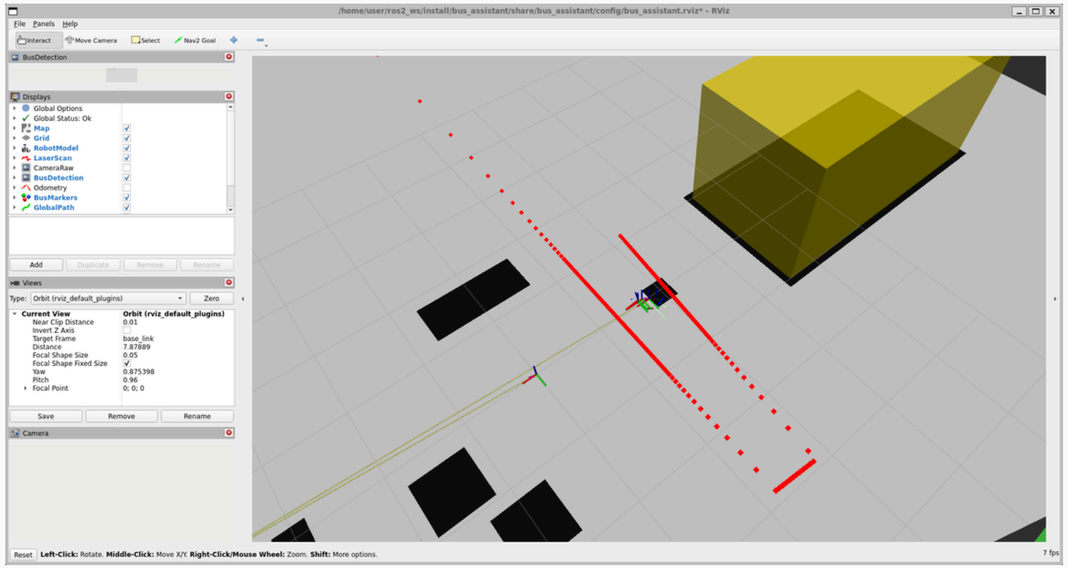
구현 방법

3.4 사용자 인터페이스 (display_node.py)
- Tkinter 다크 테마 (배경 #1a1a2e)
- 4개 색상별 버스 선택 버튼 (호버 효과)
- 로봇 상태 + 안내 메시지 색상 피드백 표시
- ROS 스핀 스레드 / GUI 스레드 간 메시지 큐 통신
- 한국어 폰트: NanumGothic, Noto Sans CJK KR



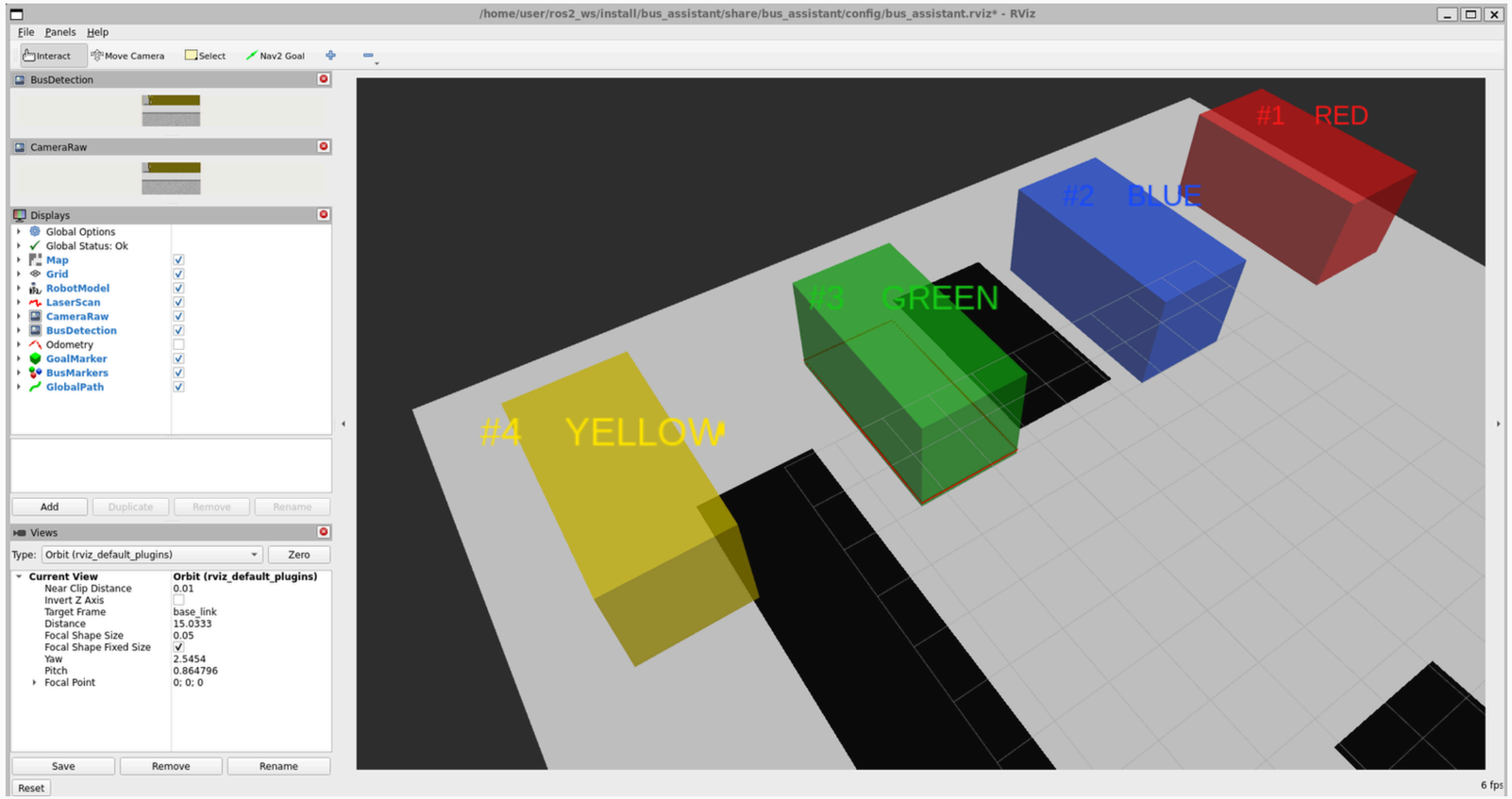


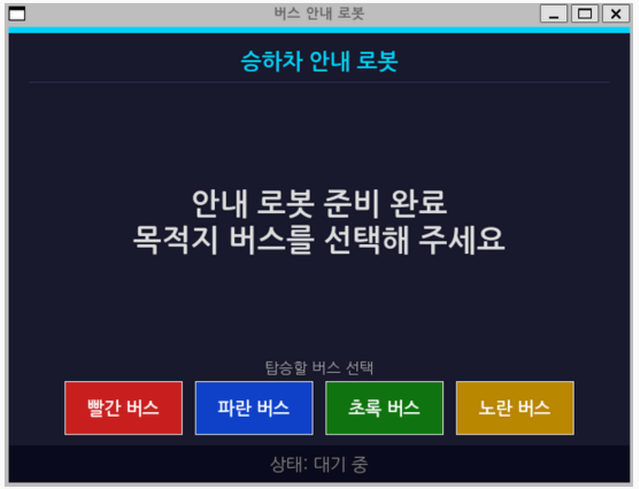
3.5 로봇 모델 (URDF)
- 구동계: 좌우 바퀴 (지름 0.14m, 간격 0.29m) + 캐스터 휠
- 센서: 전방 하향 카메라 (0.2 rad) + 360° 2D LiDAR
- Gazebo: DiffDrive, GPU LiDAR, RGB 카메라 플러그인



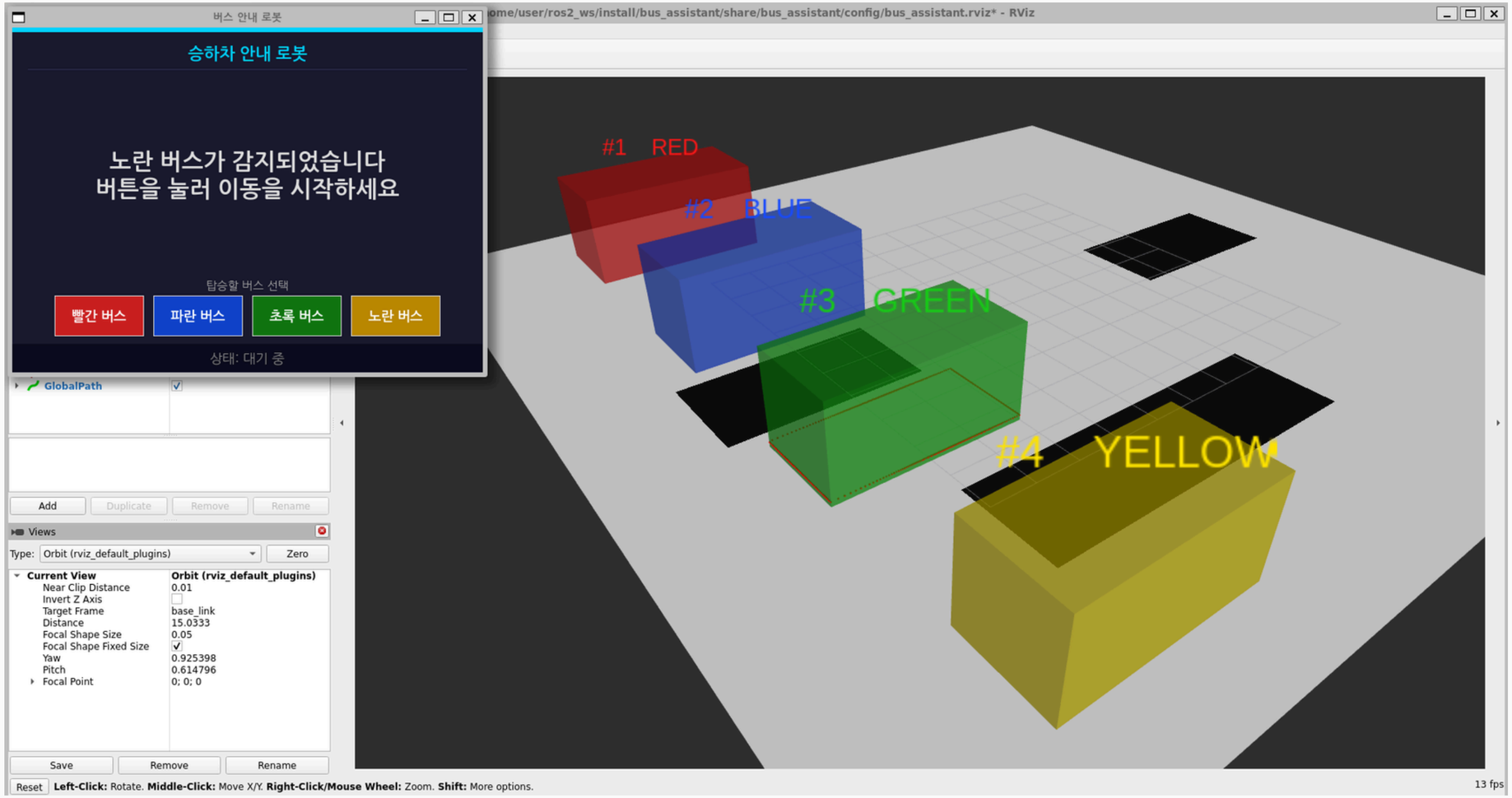
3.6 시뮬레이션 환경 (bus_stop.sdf)
- 맵: 30x30m 평지
- 버스 4개 (4x2x1.5m 직육면체)
  Red (5.5, 0) / Blue (3.0, 5.5)
  Green (0.0, 5.5) / Yellow (-5.5, 0)
- 사전 지도: 0.05m 해상도, 400x400px 점유 격자

결과


/home/user/ros2_ws/install/bus_assistant/share/bus_assistant/config/bus_assistant.rviz* - RViz
File Panels Help
Interact
Move Camera
Select
Nav2 Goal
BusDetection
CameraRaw
Displays
Global Options
Global Status: Ok
Map
Grid
RobotModel
LaserScan
CameraRaw
BusDetection
Odometry
GoalMarker
BusMarkers
GlobalPath
Add
Duplicate
Remove
Rename
Views
Type: Orbit (rviz_default_plugins)
Zero
Current View
Orbit (rviz_default_plugins)
Near Clip Distance
0.01
Invert Z Axis
Target Frame
base_link
Distance
15.0333
Focal Shape Size
0.05
Focal Shape Fixed Size
Yaw
2.5454
Pitch
0.864796
Focal Point
0; 0; 0
Save
Remove
Rename
Reset
#1 RED
#2 BLUE
#3 GREEN
#4 YELLOW
6 fps


버스 안내 로봇
승하차 안내 로봇
안내 로봇 준비 완료
목적지 버스를 선택해 주세요
탑승할 버스 선택
빨간 버스
파란 버스
초록 버스
노란 버스
상태: 대기 중

데모 영상


버스 안내 로봇
승하차 안내 로봇
노란 버스가 감지되었습니다
버튼을 눌러 이동을 시작하세요
탑승할 버스 선택
빨간 버스
파란 버스
초록 버스
노란 버스
상태: 대기 중
ome/user/ros2_ws/install/bus_assistant/share/bus_assistant/config/bus_assistant.rviz* - RViz
#1 RED
#2 BLUE
#3 GREEN
#4 YELLOW
GlobalPath
Add
Duplicate
Remove
Rename
Views
Type: Orbit (rviz_default_plugins)
Zero
Current View
Orbit (rviz_default_plugins)
Near Clip Distance
0.01
Invert Z Axis
Target Frame
base_link
Distance
15.0333
Focal Shape Size
0.05
Focal Shape Fixed Size
Yaw
0.925398
Pitch
0.614796
Focal Point
0; 0; 0
Save
Remove
Rename
Reset
Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click/Mouse Wheel: Zoom. Shift: More options.
13 fps

어려웠던 점 또는 개선 방향

01 Nav2 실행 순서 및 타이밍

문제

Nav2 준비 완료 전에 목표 좌표를 보내면 주행 명령이 처리되지 않음

해결

런치 파일에 15초 지연 TimerAction 적용으로 Nav2 초기화 대기

02 GUI 한국어 폰트 표시

문제

운영체제별 한글 폰트 차이로 안내 문구가 깨지거나 미표시

해결

NanumGothic, Noto Sans CJK KR 등 순차 확인하는 폰트 폴백 로직 적용

03 카메라 색상 인식 오탐

문제

HSV 방식이 조명/그림자 변화에 민감하여 색상 오감지 발생

해결

최소 감지 면적 설정 + 모폴로지 Opening/Closing 필터로 노이즈 제거

어려웠던 점 또는 개선 방향

01 딥러닝 기반 버스 인식

HSV 임계값 방식은 조명 변화에 취약

YOLO 등 객체 탐지 모델 적용
버스, 표지판, 노선번호 동시 인식
실제 환경에 강건한 인식 구현

02 사용자 인터페이스 개선

Tkinter 버튼 방식은 기본 기능만 제공

웹 기반 UI / 터치스크린 전환
승차권, 목적지 직관적 선택
음성 안내로 교통약자 접근성 향상

03 다중 로봇 운영 확장

현재 단일 로봇 + 단일 사용자 구조

ROS2 네임스페이스로 다중 로봇 운영
구역별 담당 또는 가까운 로봇 배정
동시 다수 이용자 안내 가능

04 실제 로봇 적용

현재 Gazebo 시뮬레이션에서만 검증

TurtleBot3 등 실제 로봇 적용 가능
RGB 카메라 + 2D LiDAR 장착
동일 ROS2 노드 구조 재사용

The End